SAIL-PROXY Developer Guide - Chapter 13

st-gr

2025-01-28

SAIL-PROXY Developer Guide

Multi-provider AI Gateway for SAP AI Core - Developer Documentation **Author:** *st-gr*

<< Previous Chapter | Content Table

Plugin System Development

The SAIL-PROXY gateway includes a sophisticated plugin system that allows developers to intercept and modify requests and responses dynamically. This chapter provides comprehensive guidance for developing, testing, and deploying custom plugins.

Overview

The plugin system supports intercepting requests at multiple points in the processing pipeline:

- **`before`**: Execute before making the upstream request (can short-circuit)
- **`after`**: Execute after receiving the upstream response (can modify response)
- **`stream`**: Execute on each chunk of streaming responses (can modify chunks)
- **`error`**: Execute when an error occurs during request processing (can customize error handling)

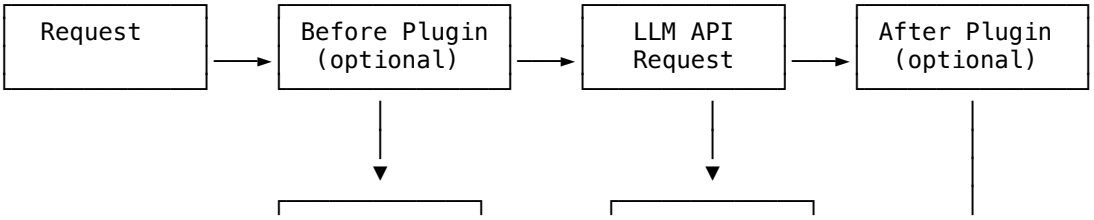
Supported Providers

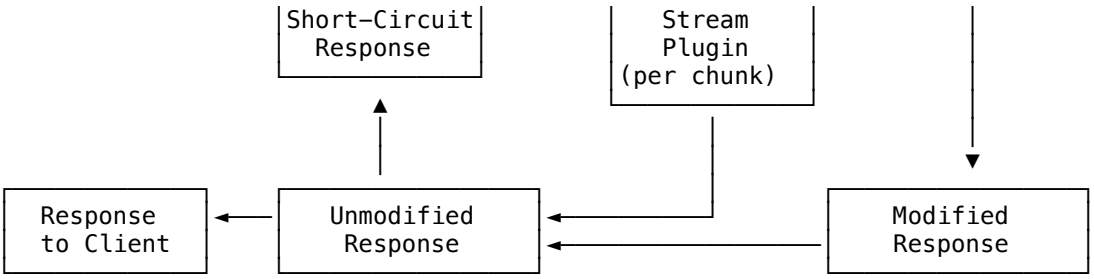
The plugin system supports all major provider endpoints:

Provider	Endpoint Path	Subpath Values
AWS Bedrock	`/aws-bedrock/model/{modelId}`	`invoke`, `invoke-with-response-stream`, `converse`, `converse-stream`
Anthropic	`/anthropic`	`invoke`, `invoke-with-response-stream` (streaming)
OpenAI	`/openai`	`invoke` (chat completions)

Plugin Architecture

Plugin Flow Diagram





Core Implementation Files

Plugin System Components:

- **Plugin Loader** (`services/gateway/src/services/pluginLoader.ts`): Discovers and loads plugins from the filesystem
- **Plugin Executor** (`services/gateway/src/services/pluginExecutor.ts`): Executes plugin hooks during request processing
- **Plugin Directory** (`services/gateway/src/plugins/`): Contains plugin implementations

Example Plugins:

- `awsBedrockResponseCache.ts` - Shows caching implementation
- `removeEmptyAssistantMessages.ts` - Request preprocessing example
- `stripCacheControlScope.ts` - Claude Code compatibility fix (strips unsupported cache_control fields)
- `repairToolBlocks.ts` - Repairs compressed tool_use/tool_result blocks missing required fields (id, name, tool_use_id)
- `resizeOversizedImages.ts` - Resizes images exceeding per-model dimension limits in multi-image requests
- `pseudonymization/` - Detects and masks PII/secrets in outbound requests and unmasks them in responses; activation and per-category masking are configurable in `api_config.json`. Precision is tuned with four optional keys in the same block: `min_confidence` and `thresholds` (how much evidence a value needs before it is masked, globally and per category), `allowlist` (`terms`, case-sensitive literals, and `patterns`, regex sources the gateway anchors to the whole value — a listed value is exempt no matter what the score says), and `saturation_warn` (report-only: above that many distinct masked values the request logs one warning and the usage SIEM event carries `saturated: true`, but every value is still masked). See `services/gateway/src/plugins/pseudonymization.md` for activation methods and category toggles, and its "Tuning pseudonymization precision" section for what raising each key costs, how to read the saturation report, and how to re-baseline with the `services/gateway/test/pseudonymization-precision/` harness (the CI regression gate for every detector change).

Creating a Plugin

1. Basic Plugin Structure

Create a JavaScript/TypeScript file in the `services/gateway/src/plugins/` directory:

```
// services/gateway/src/plugins/myPlugin.ts
import { Request, Response } from 'express';

interface PluginHandlerParams {
  req: Request;
  res: Response;
  utils: PluginUtils;
  upstreamResponse?: any;
  chunk?: Buffer;
}

interface PluginUtils {
```

```
  logger: {
    info: (message: string) => void;
    debug: (message: string) => void;
    error: (message: string) => void;
  };
  sseWriter: (res: Response, events: SSEEvent[]) => Promise<void>;
}

interface SSEEvent {
  event: string;
  data: any;
}

interface PluginRule {
  id: string;
  match: string[];  // MUST be empty - matching in api_config.json
  strategy: 'before' | 'after' | 'stream' | 'error';
  handler: (params: PluginHandlerParams) => Promise<any>;
}

const pluginRules: PluginRule[] = [
  {
    id: "myUniquePluginId",
    match: [],  // Match rules moved to api_config.json
    strategy: "before",
    handler: async ({ req, res, utils }) => {
      utils.logger.info('Plugin executing');
      // Plugin logic here
      return { stop: false };  // Continue to upstream
    }
  }
];

export = pluginRules;
```

2. Plugin Configuration

Configure your plugin in `api_config.json`:

```
{
  "hooks": {
    "definitions": {
      "size:min10kb": {
        "type": "header",
        "name": "content-length",
        "from": 10240,
        "to": 999999999
      },
      "header:x-app=cli": {
        "type": "header",
        "name": "x-app",
        "equals": "cli"
      },
      "anthropic:all": {
        "type": "url-regex",
        "regex": "anthropic",
        "flags": "i",
        "desc": "Match any URL containing 'anthropic'"
      }
    }
  },
  "models": {
    "overrides": {
      "anthropic--claude-4.5-haiku--deployed": {
        "hooks": {
```

```
          "invoke-with-response-stream": [
            {
              "request": {
                "match": ["size:min10kb", "header:x-app=cli"],
                "callback": {
                  "id": "myUniquePluginId"
                }
              }
            }
          ]
        }
      }
    }
  }
}
```

3. Hook Definition Types

The `hooks.definitions` section supports various matching criteria:

Content Length Range

```
"size:min10kb": {
  "type": "header",
  "name": "content-length",
  "from": 10240,
  "to": 999999999,
  "desc": "Cache requests with content-length >= 10 KB"
}
```

Header Value Matching

```
"header:x-app=cli": {
  "type": "header",
  "name": "x-app",
  "equals": "cli"
}
```

JSON Path Matching

```
"payload:maxTokens512": {
  "type": "json-path",
  "path": "$.max_tokens",
  "equals": 512
}
```

JSON Path Regex Matching

```
"tools:hasFileSearch": {
  "type": "json-path-regex",
  "path": "$.tools",
  "regex": "file_search",
  "desc": "Match requests containing a file_search tool"
}
```

URL Regex Matching

```
"anthropic:all": {
  "type": "url-regex",
  "regex": "anthropic",
  "flags": "i",
```

```
  "desc": "Match any URL containing 'anthropic'"
}
```

Matcher semantics: `equals` and `regex`

The `equals` and `regex` keys are compared differently depending on the rule `type`. The schema also constrains which keys a given type accepts, so a key that a type never reads is rejected at validation time rather than sitting inert in the config.

`equals` — the value the addressed thing must equal:

- **`header`** rule — compared on **media type**, not by strict equality: everything before the first `;` is trimmed and lowercased, so `"application/json"` matches a request sending `Content-Type: application/json; charset=utf-8`. This is deliberate: the strict comparison it replaced silently excluded charset-appending clients (OkHttp, .NET `JsonContent`, older axios), which for `header:contentTypeJson` meant those requests **bypassed pseudonymization on every endpoint**. An expected value that itself contains a `;` restores exact matching, so a rule can still pin a specific charset when it means to.
- **`json-path`** rule — **strict** equality (`===`) against the value the path resolves to in the request body, so `512` (number) and `"512"` (string) are different rules.
- **Omitted** — a `header` rule falls through to a `from`/`to` numeric-range check and finally to a bare presence check (the header exists with any value); a `json-path` rule matches when the path resolves to anything at all.
- **Rejected** on `json-path-regex` and `url-regex` — those match by regular expression and never read `equals`, so the schema forbids the key on them (`not: { required: ["equals"] }`).

`regex` — a JavaScript regular-expression **source** string (paired with optional `flags`):

- **`json-path-regex`** rule — tested against whatever the path resolved to: a **string** directly; an **array** element by element, stringifying a non-string element first and matching if **any** element matches; **any other object** against its `JSON.stringify` form.
- **`url-regex`** rule — tested against `req.url`, i.e. the request **path and query string** as the gateway sees them, **not** an absolute URL. An expression anchored with `^https?://` can therefore never match.
- Each expression is compiled once and cached per rule id, source and flags. An expression the regex engine rejects is caught and treated as **no match** (never a crash).
- An **empty** source is rejected by the schema (`minLength: 1`); write a real expression.

Plugin Strategies

Before Strategy

Executes before the upstream LLM call. Can short-circuit the request:

```
async function beforeHandler({ req, res, utils }: PluginHandlerParams): Promise<{stop: boolea
        n, response?: any}> {
  // Example: Cache check
  const cacheKey = generateCacheKey(req);
  const cachedResponse = await cache.get(cacheKey);

  if (cachedResponse) {
    utils.logger.info('Serving from cache');

    // Send cached response directly
    await utils.sseWriter(res, cachedResponse.events);
    return { stop: true };  // Short-circuit
  }

  return { stop: false };  // Continue to upstream
}
```

After Strategy

Executes after the upstream LLM call. Can modify the response:

```
async function afterHandler({ req, res, upstreamResponse, utils }: PluginHandlerParams): Prom
        ise<any> {
  if (!upstreamResponse) {
    return { content: [] };
  }

  try {
    // Example: Add custom metadata
    if (upstreamResponse.content && Array.isArray(upstreamResponse.content)) {
      upstreamResponse.metadata = {
        processedAt: new Date().toISOString(),
        pluginVersion: '1.0.0'
      };
    }

    return upstreamResponse;
  } catch (error) {
    utils.logger.error(`Error in after handler: ${error}`);
    return upstreamResponse;
  }
}
```

Stream Strategy

Executes on each streaming chunk:

```
async function streamHandler({ chunk, utils }: PluginHandlerParams): Promise<{chunk: Buffer,
        capturedEvents?: any[]}> {
  try {
    // Example: Content filtering
    const chunkStr = chunk.toString();
    const filtered = chunkStr.replace(/badword/gi, '***');

    return {
      chunk: Buffer.from(filtered),
      capturedEvents: [] // For caching plugins
    };
  } catch (error) {
    utils.logger.error(`Error in stream handler: ${error}`);
    return { chunk };
  }
}
```

Error Strategy

Executes when an error occurs:

```
async function errorHandler({ req, error, utils }: PluginHandlerParams): Promise<{stop: boole
        an, response?: any}> {
  utils.logger.error(`Handling error: ${error.message}`);

  // Example: Custom error response
  if (error.code === 'RATE_LIMIT_EXCEEDED') {
    return {
      stop: true,
      response: {
        error: {
          type: 'rate_limit_error',
          message: 'Please try again in a moment'
        }
```

```
      }
    };
  }

  return { stop: false };  // Use default error handling
}
```

Advanced Plugin Patterns

Multiple Plugin Rules with Same ID

You can define multiple plugin rules with the same ID but different strategies:

```
module.exports = [
  {
    id: "awsBedrockResponseCache",
    match: [],
    strategy: "before",
    handler: async ({ req, res, utils }) => {
      // Check cache and return cached response if available
      // Return { stop: true } if serving from cache
    }
  },
  {
    id: "awsBedrockResponseCache",
    match: [],
    strategy: "stream",
    handler: async ({ req, res, chunk, utils }) => {
      // Capture streaming data for caching
      // Return the chunk (possibly modified)
    }
  },
  {
    id: "awsBedrockResponseCache",
    match: [],
    strategy: "after",
    handler: async ({ req, res, upstreamResponse, utils }) => {
      // Save captured data to cache
      // Return the response
    }
  },
  {
    id: "awsBedrockResponseCache",
    match: [],
    strategy: "error",
    handler: async ({ req, error, utils }) => {
      // Clean up on error
      // Return { stop: true } to prevent default error handling
    }
  }
];
```

Plugin Utilities

SSE Writer

For streaming responses, use the `sseWriter` utility:

```
await utils.sseWriter(res, [
  {
    event: "message_start",
    data: {
```

```
      message: {
        id: messageId,
        type: "message",
        role: "assistant",
        content: [],
        model: "your-model-name"
      }
    }
  },
  {
    event: "content_block_delta",
    data: {
      index: 0,
      delta: {
        type: "text_delta",
        text: "Your response text"
      }
    }
  },
  {
    event: "message_stop",
    data: { type: "message_stop" }
  }
]);
```

Plugin Development Best Practices

1. Error Handling

Always wrap plugin logic in try-catch blocks:

```
async function safeHandler({ req, res, utils }: PluginHandlerParams) {
  try {
    // Plugin logic here
    return { stop: false };
  } catch (error) {
    utils.logger.error(`Plugin error: ${error.message}`);
    return { stop: false };  // Fail gracefully
  }
}
```

2. Performance Considerations

- Cache expensive operations
- Use async/await properly
- Avoid blocking the event loop
- Set reasonable timeouts

```
async function performantHandler({ req, utils }: PluginHandlerParams) {
  const startTime = Date.now();

  try {
    // Set timeout for external calls
    const result = await Promise.race([
      expensiveOperation(),
      new Promise((_, reject) =>
        setTimeout(() => reject(new Error('Timeout')), 5000)
      )
    ]);

    const duration = Date.now() - startTime;
    utils.logger.debug(`Plugin execution took ${duration}ms`);
```

```
    return { stop: false };
  } catch (error) {
    utils.logger.error(`Plugin failed: ${error.message}`);
    return { stop: false };
  }
}
```

3. Security Guidelines

- Validate all input data
- Sanitize content appropriately
- Never log sensitive information
- Use secure defaults

```
function validateInput(data: any): boolean {
  // Implement appropriate validation
  if (!data || typeof data !== 'object') {
    return false;
  }

  // Check for malicious patterns
  const dangerousPatterns = ['<script', 'javascript:', 'data:'];
  const str = JSON.stringify(data);

  return !dangerousPatterns.some(pattern =>
    str.toLowerCase().includes(pattern)
  );
}
```

Testing Plugins

1. Unit Testing

Create test files in `services/gateway/tests/plugins/`:

```
// services/gateway/tests/plugins/myPlugin.test.ts
import { describe, it, expect, jest } from '@jest/globals';
import pluginRules from '../../src/plugins/myPlugin';

describe('MyPlugin', () => {
  it('should handle before strategy correctly', async () => {
    const mockUtils = {
      logger: {
        info: jest.fn(),
        debug: jest.fn(),
        error: jest.fn()
      },
      sseWriter: jest.fn()
    };

    const rule = pluginRules.find(r => r.strategy === 'before');
    const result = await rule.handler({
      req: {} as any,
      res: {} as any,
      utils: mockUtils
    });

    expect(result.stop).toBe(false);
    expect(mockUtils.logger.info).toHaveBeenCalled();
  });
});
```

2. Integration Testing

Test plugins with real requests:

```
# Start the gateway in development mode
pnpm run dev

# Test plugin with curl
curl -X POST http://localhost:3000/anthropic/v1/messages \
  -H "Content-Type: application/json" \
  -H "x-api-key: your-test-key" \
  -H "x-app: cli" \
  -d '{
    "model": "claude-haiku-4-5-20251001",
    "messages": [{"role": "user", "content": "test"}],
    "max_tokens": 512
  }'
```

3. Debugging Plugins

Enable debug logging to troubleshoot plugins:

```
DEBUG=true pnpm run dev
```

Add debug statements in your plugin:

```
async function debugHandler({ req, utils }: PluginHandlerParams) {
  utils.logger.debug('Plugin input:', {
    url: req.url,
    headers: req.headers,
    body: req.body
  });

  // Plugin logic...

  utils.logger.debug('Plugin output:', { result });
  return result;
}
```

Plugin Deployment

1. Development Deployment

For local development:

1. Place plugin file in `services/gateway/src/plugins/`
2. Update `api_config.json` with hook configuration
3. Restart the gateway service

2. Production Deployment

For production environments:

1. Test thoroughly in development
2. Code review plugin implementation
3. Update configuration via admin API
4. Deploy with rolling restart strategy

3. Plugin Distribution

Consider creating reusable plugins as npm packages:

```
{
  "name": "@yourorg/sail-proxy-plugin-cache",
  "version": "1.0.0",
  "main": "dist/index.js",
  "peerDependencies": {
    "@sail-proxy/gateway": "^1.0.0"
  }
}
```

Example: Complete Caching Plugin

Here's a complete example implementing response caching:

```
// services/gateway/src/plugins/responseCache.ts
import { Request, Response } from 'express';
import Redis from 'ioredis';

const redis = new Redis(process.env.REDIS_URL || 'redis://localhost:6379');

interface PluginHandlerParams {
  req: Request;
  res: Response;
  utils: any;
  upstreamResponse?: any;
  chunk?: Buffer;
}

// Before handler - check cache
async function checkCache({ req, res, utils }: PluginHandlerParams) {
  try {
    const cacheKey = `cache:${req.method}:${req.url}:${JSON.stringify(req.body)}`;
    const cached = await redis.get(cacheKey);

    if (cached) {
      utils.logger.info('Cache hit', { cacheKey });
      const response = JSON.parse(cached);

      if (response.streaming) {
        await utils.sseWriter(res, response.events);
      } else {
        res.json(response.data);
      }

      return { stop: true };
    }

    // Store cache key for later use
    req.cacheKey = cacheKey;
    return { stop: false };

  } catch (error) {
    utils.logger.error('Cache check failed', { error: error.message });
    return { stop: false };
  }
}

// After handler - store in cache
async function storeCache({ req, upstreamResponse, utils }: PluginHandlerParams) {
  try {
    if (req.cacheKey && upstreamResponse) {
```

```
      const cacheData = {
        data: upstreamResponse,
        streaming: false,
        cachedAt: new Date().toISOString()
      };

      // Cache for 1 hour
      await redis.setex(req.cacheKey, 3600, JSON.stringify(cacheData));
      utils.logger.info('Response cached', { cacheKey: req.cacheKey });
    }

    return upstreamResponse;

  } catch (error) {
    utils.logger.error('Cache store failed', { error: error.message });
    return upstreamResponse;
  }
}

const pluginRules = [
  {
    id: "responseCache",
    match: [],
    strategy: "before" as const,
    handler: checkCache
  },
  {
    id: "responseCache",
    match: [],
    strategy: "after" as const,
    handler: storeCache
  }
];

export = pluginRules;
```

Troubleshooting

Common Issues

1. **Plugin not loading**: Check file path and export format
2. **Match conditions not working**: Verify `hooks.definitions` in `api_config.json`
3. **Handler errors**: Check error logs and add proper error handling
4. **Performance issues**: Profile plugin execution time

Debug Commands

```
# Check loaded plugins
curl http://localhost:3000/api/debug/plugins

# View plugin execution logs
tail -f logs/gateway.log | grep Plugin

# Test specific hook definitions
curl -X POST http://localhost:3000/api/debug/hooks \
  -d '{"url": "/anthropic/v1/messages", "headers": {"x-app": "cli"}}'
```

For more information on gateway architecture, see Chapter 3: Gateway Service.